RESEARCH DOCUMENT

Researcher: Vilius Bučinskas

Fontys University of Applied Sciences
Smart Mobile Minor

Contents

What is this research about? 2
Why did we need the results of this research? 2
Approach 2
Results 3
What is hybrid development? 3
Advantages and Disadvantages of using Hybrid development 4
Most popular Hybrid mobile frameworks 6
Flutter 6
Comparison / Pros and Cons 7
Flutter 7
Ionic 8
React Native 8
Apache Cordova 9
Prototyping /POC's 10
Conclusion 16
My case for the project: 16
References 17

What is this research about?

In this research, we will be looking at the most popular cross-platform mobile development frameworks in 2021. We will investigate the top 4 most used frameworks and compare them and try to create a proof of concept.

For this we will be using literature study method and available product analysis to find out which hybrid framework is most used in 2021. And we will also get to know if it's possible to develop an entire application using hybrid framework only and if it's the most optimized way to do it.

Why did we need the results of this research?

The need for this research comes when business owners decide to switch from native to hybrid development, the major challenge they often encounter is choosing the right mobile technology. Transitioning from native to hybrid might be a tricky task and It's not always worth doing so. This research is a guidance for those who aim to transition from native to hybrid development and will assist in making the right choice for technology.

Approach

Main Research Question:

- Which technology to choose when transitioning from native to hybrid mobile development in 2021?

Sub-questions:

- What is hybrid development?
- What advantages and disadvantages does hybrid development have?
- What are the most popular hybrid development frameworks in 2021?
- What are all the pros and cons for each of those frameworks?

- Which framework is the easiest to start working with?

To give form to the approach to the research, the DOT framework will be applied.

Available product analysis

Literature study

To get to know more about hybrid mobile development, a literature study is needed to gather all information which is on the internet. Additionally, documentation on how to work with each framework is also gathered by reading literature.

Community research

Research into the Cross-platform developers' community may give some more insight about each framework and other developers experience with it.

Prototyping

Prototyping will be conducted to know how it feels to work with the frameworks and to conclude my personal experience working with them

Results

What is hybrid development?

"Hybrid mobile development" or "Cross-platform mobile development" is the creation of software applications that are compatible with multiple mobile operating systems. Some hybrid development software requires a separate build for each platform, but some can be directly run on any platform without special preparation.

Hybrid applications have a native look and feel to them because of the combination of native code with independent code that can run on multiple platforms. Developers write code once and then reuse it, allowing a product to be released quickly.

Advantages and Disadvantages of using Hybrid development

Multiplatform mobile development is continuously evolving because of the emerging technologies becoming more dynamic and attractive for developers.

Generally, there are main aspects in which native and hybrid app development differ:

Development time: This is where hybrid apps shines. Native apps take significantly more time to develop because separate teams need to develop for different platforms. Hybrid code can be duplicated across several platforms.

Development cost: Same logic can be used here. The more development time and personnel the bigger development cost.

Code reusability: Developers just need to code once and the codebase can be reused to deploy the app to multiple platforms. The shared codebase also allows you to add another platform at a later point without too much effort.

Easy Deployment and Maintenance: Since there is a single codebase, the developers only need to write and maintain just a single source code. This translates to easy and fast deployment, maintenance, updates and bug fixes.

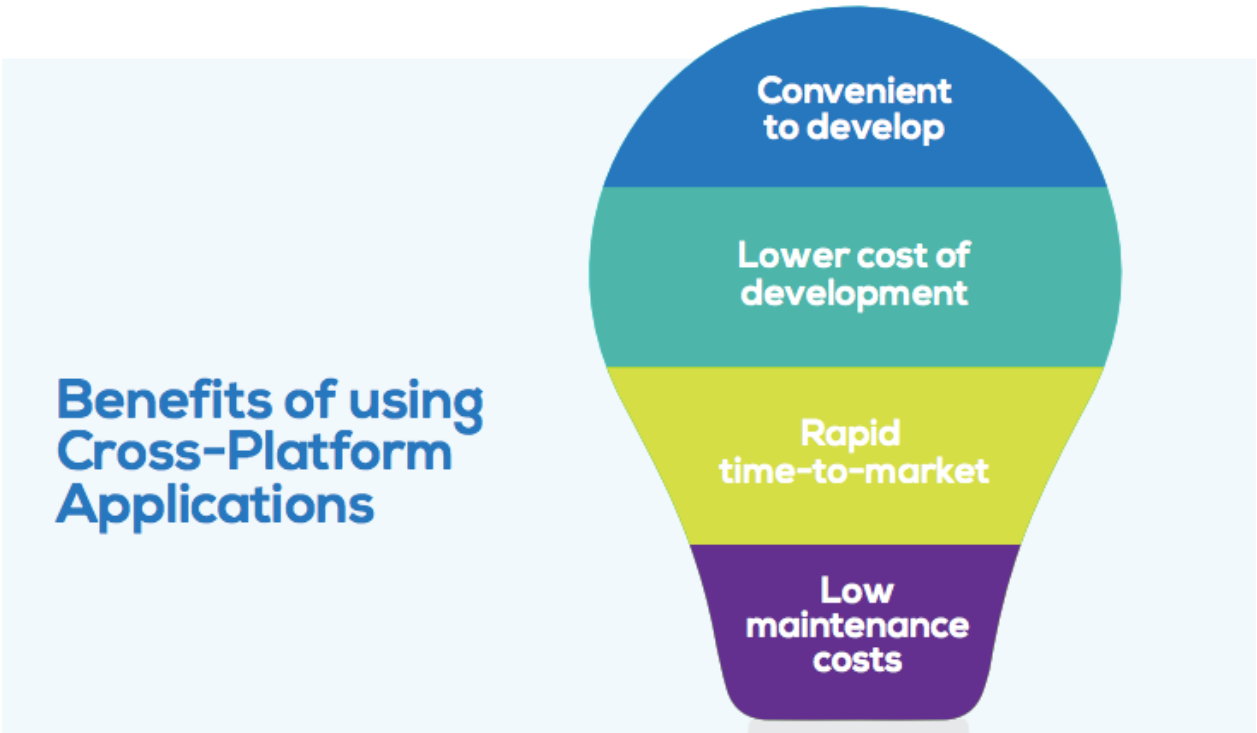
Performance: Native apps preform better and are more responsive than hybrid apps, this is mostly because native code interacts directly with the device resources.

Product lifetime: Native development is much safer for projects that are meant for long term because the framework are more stable and there are no changes overnight.

Debugging: Hybrid does not offer the same application debugging capabilities.

UI/UX design: Hybrid development is also behind in the UI/UX experience. Developing in native grants unrestricted access to UI/UX components. Native applications usually deliver more stunning visuals.

So, what is the right choice? Unless you're making a high-performance game or other similar software, hybrid app development may be the best option because it allows for a faster development process, lower costs, and platform compatibility. While they do require a little more knowledge to put the solution together, as well as the use of a UI library to aid in the proper presentation of your user interface, those challenges are well-known and easily solved with the right framework.



Most popular Hybrid mobile frameworks

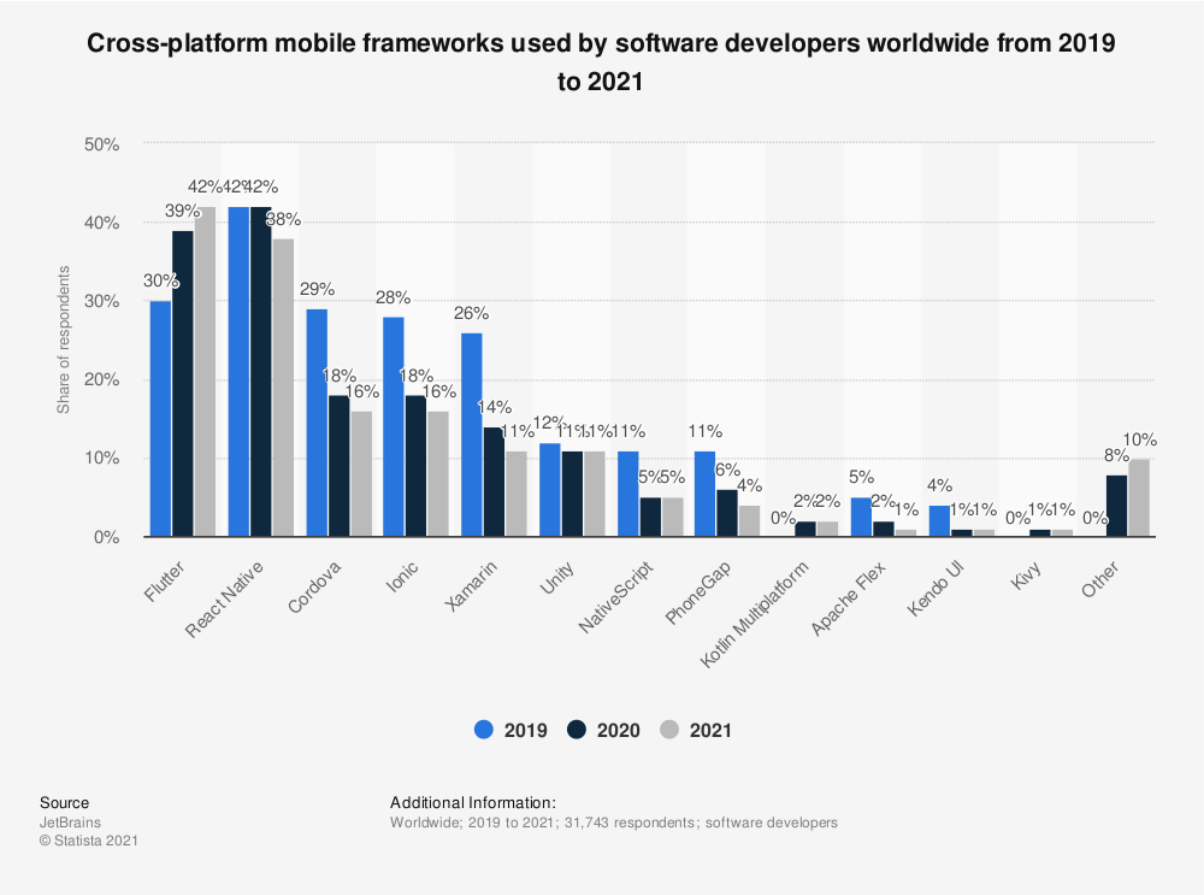


Figure 1 – The graph shows the most used hybrid mobile frameworks by software developers from 2019 to 2021.

As shown in Figure 1, the survey results come from almost 32 thousand software developers all over the globe. For this research I will assume that this data is accurate.

Flutter

It was developed and released by Google in 2017, flutter is a popular open source and free hybrid framework. It uses Dart programming language to develop applications for Android, iOS, Windows, Mac, Linux and Web from a single codebase.

It takes first place in 2020 and 2021 as the most used cross-platform mobile framework.

The Stack Overflow Developer Survey Results 2021 Flutter was listed second as most loved framework.

React-Native

It was launched in 2015, React Native is Facebook open-source cross-platform mobile app development framework. It uses JavaScript and React.js libraries to build apps for Android, iOS, Mac, Windows, and the Web.

React-Native comes second from the list of most used cross-platform frameworks.

Cordova

Apache Cordova was released back in 2009. It is an open-source framework that utilizes HTML, CSS and JavaScript to build hybrid web apps for Android, iOS and Windows.

Ionic

Released in 2013, It is an open-source framework that leverages web technologies such as HTML, CSS and JavaScript with integrations for frameworks such as Angular, React and Vue to build for hybrid mobiles, desktops, and PWA's.

In 2020 as well as in 2021 Cordova and Ionic share third and fourth place for the most used hybrid frameworks.

Comparison / Pros and Cons

Now that we have a clearer idea of the most used hybrid mobile frameworks in 2021, let's examine how much of a difference choosing either framework can be.

Flutter

Pros:

- **Simplicity:** Using Flutter is significantly easier than learning all the languages you need to develop an app both on Android and iOS.
- **Great SDK tools:** Developing your app with Flutter means you'll have many tools at your disposal, like the built-in hot reload

feature that allows you to see changes in your app instantly without having to recompile it.

Cons:

- **It's constantly changing:** This makes it harder to keep up with all the updates the framework goes through.
- **Flutter apps are large:** Due to their built-in widgets, Flutter apps are larger than their native counterparts. Since users have limited memory on their devices, you should always take this into account.

Ionic

Pros:

- **Support for different technologies:** Starting from Ionic 4, you can use any frontend framework you want to create your mobile apps.
- **Great for beginners:** If you know HTML, CSS, and JavaScript, you can start building your mobile apps thanks to this fantastic tool.

Cons:

- **Not suitable for large apps:** Unfortunately, using WebView for large applications can result in a significant lack in performance that you should keep in mind.
- **May lack native plugins support:** Some native plugins aren't stable and can conflict with each other. Or they might be completely absent, forcing you to implement your solution from scratch.

React Native

Pros:

- **Performance:** The performance of apps built using React Native are incredibly close to those you would get from platform-specific code.

- **A great community:** More than 1500 contributors support this tool, and huge tech companies are using it like Airbnb, Uber, and Tesla.
- **Great for your productivity:** React Native comes with unique features like hot-reloading and Chrome developer tools for debugging your app.

Cons:

- **Few third-party components:** Due to this project being still new, there are not as many third-party components as you might expect. So sometimes, you might be forced to implement a custom solution by yourself from scratch.

Apache Cordova

Pros:

- **Build management:** Cordova creates its own config file for tracking your enabled platforms and plugins. Additionally, it provides commands such as 'cordova run android/ios'. You don't need to build your apps using different IDEs and/or command line tools for each platform, Cordova handles it for you.
- **Frameworks like ionic:** Ionic framework provides an angular code base with native-like widgets you can use. This allows you to have things like tabs, toggle buttons, reorderable lists, etc. in your web application.

Cons:

- **Performance and app size:** Well, this goes without saying. You're running your app in a web view, it's not going to reach native performance. If trying Cordova app in an average android phone, and it performed as well as most native apps from a user perspective.

Prototyping /POC's

For this part of the research, we will be investigating how hard is it to start working with each of these frameworks so I could conduct an informative conclusion therefore I'll create a proof of concept with every framework I mentioned above.

With each framework I will be making an application which shows the current time. A label with frameworks name and current hours, minutes and seconds will be displayed.

Link for all POC's: https://git.fhict.nl/I413189/hybrid_pocs

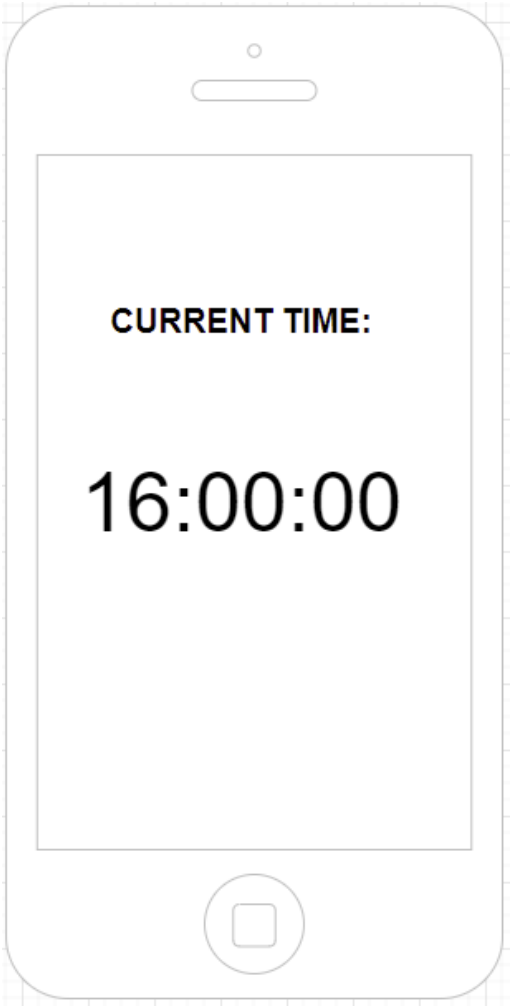


Figure 2 Wireframe of the POC

Flutter

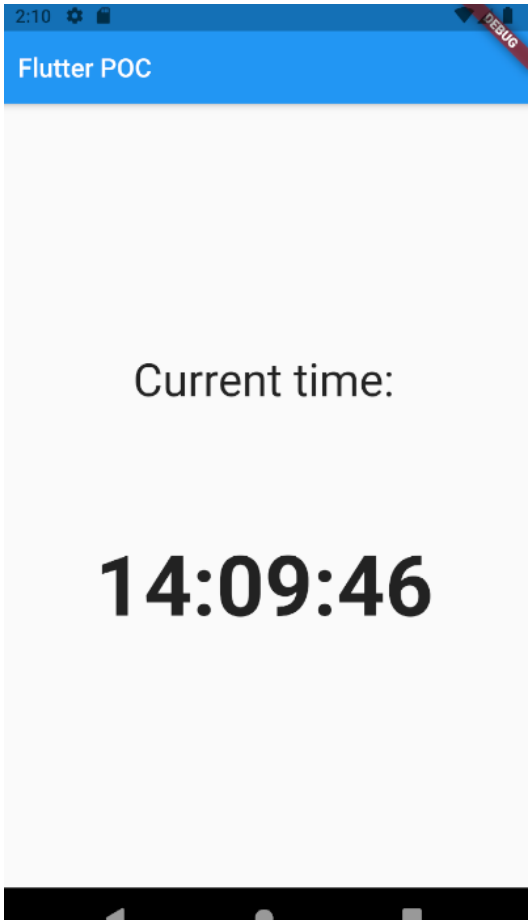


Figure 3 final POC made with flutter

- Code

```
@override
Widget build(BuildContext context) {
  DateTime now = DateTime.now();
  String formattedDate = DateFormat('kk:mm:ss').format(now);
  // This method is rerun every time setState is called, for insta
```

This is now I get the current time in flutter and with the use of a package I format it to a wished structure.

Flutter uses Dart programming language so It could slowed the studying process but from my experience It is easy to catch up.

- Time spent

In total for this POC I spent approximately 30 minutes. As I have worked with this framework before I already had it installed and knew the basic functions.

Apache Cordova

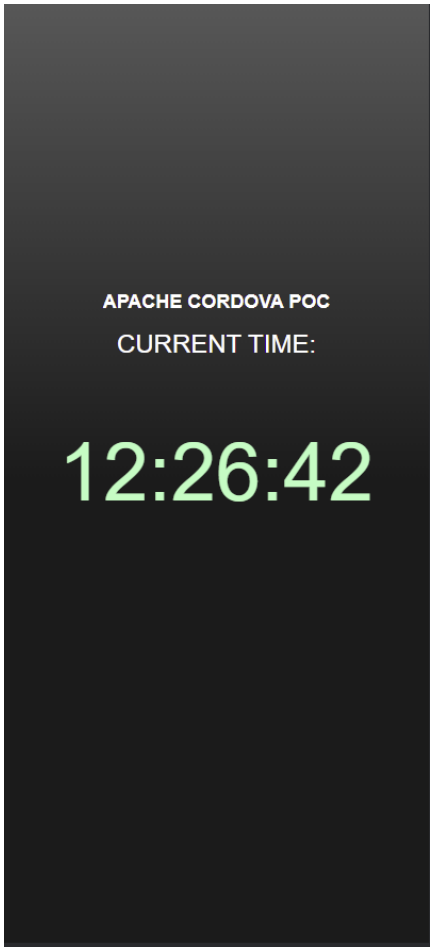


Figure 4 Final POC made with cordova

- Code:

Apatche Cordova allows to use standard web technologies like HTML, CSS and JavaScript for hybrid development thus It makes an easy learning curve for web developers or anyone who is familiarize with basic web technologies.

```
function showTime() {
    var date = new Date();
    var h = date.getHours();
    var m = date.getMinutes();
    var s = date.getSeconds();

    var time = h + ":" + m + ":" + s;
    document.getElementById("ClockDisplay").innerText = time;
    document.getElementById("ClockDisplay").textContent = time;
}
```

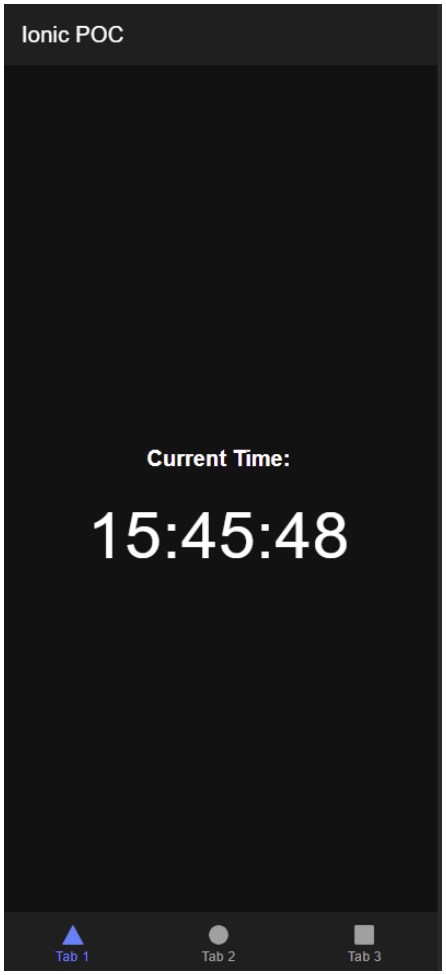
Figure 5

In the picture above (Figure 4), we can see a function needed to retrieve the current time for the proof of concept. I can reuse my JavaScript code to retrieve time and it works flawlessly with Cordova.

- Time spent:

In total for this POC I spent approximately 1 hour and 30 minutes because I had some problems with Android SDK which took time to solve, but to write this simple code and to navigate around the framework it took me 30 – 40 minutes.

Ionic



- Code

Ionic also uses standard web technologies like HTML, CSS and JavaScript to create the GUI, just like Apache Cordova.

```
export class Tab1Page {
  constructor() {}        You,
  today = Date.now();
}
```

```
  <app-explore-container name="Current Time:"></app-explore-container>
  <div id="container">
    <div class="ion-text-center">
      <ion-title size="large">{{ today | date: 'HH:mm:ss' }}</ion-title>
    </div>
  </div>
</ion-content>
```

The formatting is easy because of the built in pipes.

- Time spent

In total for this POC I spent approximately 1 hour. It was hard to navigate around the files and the documentation wasn't great.

React-Native

- Code:

```
export default class App extends Component {
  componentDidMount() {
    setInterval(
      () => this.setState({ curTime: new Date().toLocaleString() }),
      1000
    );
  }

  state = { curTime: new Date().toLocaleString() };

  render() {
```

- Time spent

In total for this POC I spent 30 minutes in total. As I have worked with this framework while doing internship so I already had it installed and knew the basics.

Conclusion

After comparing the different advantages and disadvantages of Flutter, Ionic, Cordova and React Native, you can easily make some assumptions about which one you should choose for which situations:

Use Flutter for your MVP (Minimum Viable Product): If you want to test a possible app idea, Flutter is the way to go. First, it provides hundreds of built-in widgets to make your apps look beautiful on all platforms. Second, it helps you create your product extremely fast thanks to its great SDK full of top features like hot reloading.

Use Ionic, Cordova or React Native to start competing: If you want to start working in the mobile world, these technologies are a fantastic choice. With Ionic, you can use any modern frontend framework from the start; Cordova will then wrap your app and turn it into a native product. If you or your team are really into React, then you go for the React Native route. This will allow you to extremely ship performant mobile and web apps in no time with your pre-existing JavaScript skills.

My case for the project:

I chose to make case no.1

> Visitors to a pop concert can (if allowed by the organisation of the concert) stream live video (and music) to a server with this app. Users can then log on to that server and follow the concert live on screen

Framework selection: React-Native

Appendix:

References

Borrelli, P. (be datos). *Comparing Mobile Development Platforms in 2021*. Nuskaityta iš strapi: https://strapi.io/blog/comparing-mobile-development-platforms-in-2021

Cross-platform mobile frameworks used by software developers worldwide from 2019 to 2021. (be datos). Nuskaityta iš statista: https://www.statista.com/statistics/869224/worldwide-software-developer-working-hours/

errietta. (be datos). *A first look at Cordova: pros and cons*. Nuskaityta iš erriettas blog: https://www.errietta.me/blog/apache-cordova-pros-cons/

Luetić, M. (2021 m. March 15 d.). *Cross-Platform vs. Native Mobile Development*. Nuskaityta iš decode: https://decode.agency/native-vs-cross-platform-mobile-apps/

Smith, R. (be datos). *Top 10 Best Cross Platform App Development Frameworks in 2021*. Nuskaityta iš codenameone: https://www.codenameone.com/blog/top-10-best-cross-platform-app-development-frameworks-in-2021.html